

Conformance without match rates

An auditable predicate for validating encoded tax and benefit law against reference implementations

The Axiom Foundation

2026-07-28

Abstract

Machine-readable encodings of tax and benefit law are usually validated by comparing them against an existing microsimulation model and reporting an agreement rate. We argue that an agreement rate is the wrong summary, because it counts a difference that has been traced to a documented defect in the reference implementation the same as a difference nobody has explained. The two are not comparable evidence, and a single percentage destroys the distinction precisely where a reader needs it.

We describe a validation harness that replaces the rate with a predicate. A jurisdiction is conformant when every policy the reference model simulates is either covered by a live comparison or excluded for a declared reason, no mismatch is unexplained, and no mismatch is attributed to our own encoding. Each residual difference must be classified with evidence that the build recomputes: an arithmetic reconciliation that must evaluate to the claimed value, or a citation to a defect in the reference model. Two of the five classifications never count as explained, so a residual cannot be dispositioned away by asserting that it is fine.

We report results for four jurisdiction-oracle pairs. Three are conformant. The fourth, the United States against PolicyEngine, is not — and fails on coverage rather than on disagreement, which is a materially different and more useful statement than any percentage would have produced. We give one suite decomposed in full, in which 18,791 mismatches across 3.9 million comparisons resolve into four documented causes with no remainder, and one policy that sits at 42% raw agreement while being conformant.

1 The problem with agreeing 99% of the time

Suppose you have encoded a country’s income tax in a machine-readable form and you want to know whether you got it right. The natural move is to find something that already computes the same thing — a national microsimulation model, a tax calculator, an administrative system — run both over the same population, and report how often they agree.

This is differential testing (McKeeman 1998) applied to policy, and in a field with no closed-form specification it is close to the only tool available. Statute does not come with an executable reference. The oracle problem in software testing — the difficulty of deciding whether an observed output is the correct one (Barr et al. 2015) — is unusually acute here, because the artefact under test is itself a reading of a text that reasonable people read differently. An independent implementation is the strongest oracle on offer, and comparing against one is standard practice: EUROMOD validates its baselines against external administrative aggregates and national models (Sutherland and Figari 2013), and TAXSIM has served as a reference point for United States tax calculations since the early 1990s (Feenberg and Coutts 1993).

The trouble starts at the moment of reporting. Two implementations of a large tax-benefit system never agree completely, so the comparison produces a rate, and the rate is published. But consider what a rate has to do to two very different residuals:

- A difference where the reference model is running an older release whose behaviour has since been corrected upstream, where we can name the change, reproduce the arithmetic to the cent, and link the merged fix.
- A difference nobody has looked at.

A percentage adds these together. Worse, it is monotone in the wrong direction for the reader’s purposes: a system that has investigated its residuals looks identical to one that has not, and a system whose reference model happens to be older looks worse than one compared against a model with fewer features. The number moves for reasons that have nothing to do with whether the encoding is correct.

Two observations from our own data make this concrete. In one suite, the United Kingdom’s Universal Credit compared against UKMOD, raw agreement is 42.1%. Every one of the residual differences is a documented behaviour of the reference model. Reporting 42.1% would tell a reader almost the opposite of what the evidence supports. In another, federal income tax compared against PolicyEngine, raw agreement is 99.52% — and the 18,791 differences behind that figure are not noise to be waved through but four specific, separately characterised causes, one of which is a known upstream change with a merged pull request attached. Neither number, on its own, is worth publishing.

Our claim is not that comparison is the wrong method. It is that the summary statistic is the wrong artefact, and that the right one is a predicate over classified evidence.

2 The predicate

For each jurisdiction and each reference model — we call the reference model the oracle — the harness evaluates exactly:

```
conformant    covered == in_scope
              AND unexplained_total == 0
              AND axiom_attributed_open == 0
```

The three conjuncts carry the argument.

`in_scope` is the set of policies the oracle simulates that we hold the encoding to. Every other simulated policy is excluded with a required reason, so that the accounting stays honest: an excluded policy is neither covered nor silently dropped, it is counted and shown alongside why. Exclusion reasons in current use include `input_carrying` (the oracle passes a reported value through rather than computing it), `oracle_models_repealed_law`, `oracle_models_nonstatutory_amount`, `unobservable_boundary`, and `technical`.

`covered` means an in-scope policy has a named comparison suite with a live committed report present. A suite named but not run is in scope and not covered. Coverage is evidence, not intent — this is the conjunct that prevents the harness from taking credit for planned work.

`unexplained_total` counts mismatches on covered suites carrying no explanatory classification.

`axiom_attributed_open` counts the residual that is ours to fix: classifications of kind `axiom_encoding_gap`, plus any mismatch whose classification links a still-open issue in our own encoding repositories. Differences attributed to the oracle, and differences caused by the comparison harness feeding the two systems different inputs, are explained and do not block conformance.

The split is the point. A suite can sit at 42% raw agreement and be conformant for that policy when every residual is a documented behaviour of the oracle; a suite at 99.5% is not conformant if a single difference has gone uninvestigated. The predicate is not a lower bar than a high match rate. It is a different and, in the cases that matter, considerably harder one.

3 Method

3.1 The universe is generated from the oracle, not from memory

The list of policies a jurisdiction is held to is parsed from the oracle’s own model definition rather than curated by hand. This matters because a hand-curated scope is unfalsifiable: whoever writes it decides what counts, and a policy that was never listed can never be missing.

For EUROMOD and UKMOD, the harness parses the model’s country XML for one system’s policy list and, per policy, the output variables its functions write, then cross-checks each output against the model’s own variable registry. An output present in the registry is a queryable comparison surface; a function-local intermediate is internal only, and that fact is recorded as the evidence for classifying it as an unobservable boundary rather than as a gap.

For PolicyEngine, the harness instantiates a pinned checkout’s tax-benefit system — the variable registry only, with no microdata and no simulation run — and, for each program in a declared spine, reads from the code which of the program’s bound output variables actually carry a formula. Variables that merely pass a reported input through are classified as input-carrying exclusions with per-row notes. The universe header pins the exact reference-model version read from the checkout’s own package metadata.

The consequence is that the denominator of the coverage fraction is not ours to choose. When the oracle adds a policy, the universe grows and coverage falls.

3.2 Every difference is classified, and two classifications never help

Each comparison suite has an associated dispositions file. Every mismatch must fall under an entry, and every entry declares one of five kinds:

Kind	Meaning	Counts as explained
explained_residual	Reconciles arithmetically to a documented convention or mechanism	yes
upstream_engine_gap	The oracle diverges from the source it models; filed or cited upstream	yes
bridge_artifact	The harness fed the two systems different inputs; not a defect in either	yes
axiom_encoding_gap	Our encoding is missing or wrong	no
unexplained	Explicitly recorded open investigation	no

The last two rows are what stop the scheme from being a way to talk one’s way out of a bad result. A difference that is our fault is classified — it is not hidden — but classifying it does not improve the verdict. The only way to move axiom_encoding_gap to zero is to fix the encoding.

3.3 The evidence is machine-checked

A classification without evidence is invalid. Each entry must carry a stated mechanism together with either an arithmetic reconciliation or an upstream citation. Where an entry claims arithmetic, the build parses and evaluates the expression and requires it to equal the claimed value within a stated tolerance; a classification must reconcile numerically rather than assert. Non-URL sources are repository-relative paths that must resolve. Entries may pin the specific values they explain, in which case the entry stops applying when the live values move away from the pin — so a disposition cannot silently relabel a new residual that happens to land in the same place, and an entry whose mismatch has disappeared is reported as expired rather than left to rot.

3.4 The ratchet only turns one way

Committed floors and ceilings record, per jurisdiction, that coverage may only rise and that unexplained and self-attributed counts may only fall. The check refuses a regression. The ratchet is advanced deliberately after a genuine improvement and is never re-pinned by automation, which keeps it a commitment rather than a running total.

3.5 Covered verdicts are reproducible outside the harness

For each covered suite, a committed composition records the runnable program the harness assembles — the import set and the repository-relative files, the query entity, the supplied-input surface, and the bridge from oracle inputs to encoding inputs. A reader can rebuild the comparison without the harness, which is what makes “covered” checkable rather than merely asserted.

4 Results

Values below are read from the committed scoreboard on 25 July 2026. Coverage counts policies; the three explained kinds are counted in mismatches.

Jurisdiction	Oracle	Covered	Unexplained	Ours, open	Conformant
Belgium	EUROMOD J2.0 / BE_2025	23 / 23	0	0	yes
United Kingdom	UK- MOD_PUB- LIC B2026.03 / UK_2026	21 / 21	0	0	yes
United Kingdom	policyengine-uk 2.89.2	23 / 23	0	0	yes
United States	policyengine-us 1.767.3	34 / 127	0	0	no

The United States row is the informative one. It fails the predicate, and it fails on the first conjunct: 93 in-scope policies have no live comparison suite. Across the 34 that do — scored from 23 suites totalling 3,997,891 comparisons — nothing is unexplained and nothing is attributed to our encoding. That is a precise and unflattering statement of where the work stands, and it is the statement a percentage would have hidden. Any single rate over the covered subset would have read as a claim about the whole system.

4.1 One suite, decomposed

Federal income tax against PolicyEngine, over every tax unit in a pinned microdata artefact, with the oracle pinned to the release that artefact was built with: 3,881,635 comparisons, 3,862,844 matches, 18,791 mismatches, raw agreement 99.52%.

The dispositions file characterises all 18,791 — not a sample:

	Count	Concept	Kind	Cause
	16,660	eitc	upstream	Earned-income input composition. The pinned oracle release predates an upstream change that split partnership and S-corporation income into separate inputs; the encoding follows 26 U.S.C. §32(c)(2)(A) and §1402(a). Diverges in both directions — 10,008 rows high, 6,652 low — the two halves of that split.
	2,118	tax_before_credits	reconciled	Bracket-boundary rounding. Every row within \$5.83, on values up to \$2.79M.
	8	capital_gain	reconciled	Floating-point noise. Every row within \$3.25.
	5	ctc	reconciled	Excess-AGI phase-out rounding. Every row exactly \$50.

These sum to 18,791, which is every mismatch in the report. Unexplained: zero.

The EITC row is worth dwelling on, because it is the case the predicate is built for. Sixteen thousand differences is not a rounding artefact; it is a real, systematic divergence affecting the largest refundable credit in the United States tax code. Under a match-rate regime it would either drag the headline number down or, more likely, be quietly excluded as a known issue. Here it is carried in full, attributed, and linked to the upstream change that explains it — and because it is classified as an oracle gap rather than an encoding gap, the claim being made is falsifiable: a reader who thinks we have the statute wrong knows exactly which provision to check and which rows to pull.

4.2 A conformant policy at 42% agreement

The United Kingdom’s Universal Credit suite compares 19 cases and matches 8, for raw agreement of 42.1%. Every residual is a documented behaviour of the reference model. The policy is conformant.

We include this because it is the case that most embarrasses the match rate. Any publication regime that leads with agreement percentages either suppresses this row or reports a number that will be read as a failure. Neither is a description of the evidence. Two further United Kingdom policies sit at 0% raw agreement and are likewise conformant: in both, the oracle computes a payable amount that our comparison surface reaches through a different, documented route.

5 What the predicate does not show

It does not establish correctness. Conformance says that two independent implementations agree, or that their disagreements are accounted for. Where both read a provision the same wrong way, the predicate is silent. This is the irreducible limit of differential testing against a single oracle (Barr et al. 2015), and no amount of classification discipline removes it. It is the reason coverage against multiple independent oracles matters more than depth against one.

It does not speak to unmodelled policy. The universe is the oracle’s simulated surface. A provision neither system models is invisible to the predicate, and no count on the scoreboard will reveal it.

Coverage is the live limit, not accuracy. Three of our four pairs are conformant, and the fourth is short by 93 policies. A reader who takes the three conformant rows as a claim about the United States has

misread them, which is why the coverage fraction is reported in the same table rather than in a footnote.

Attribution is a judgement, made by an interested party. We classify a difference as the oracle’s rather than ours. That judgement is checkable — the mechanism is stated, the arithmetic is recomputed, the upstream link resolves — but it is ours to make, and a reader should treat the `upstream_engine_gap` column as a claim to audit rather than a fact to accept. The evidence requirements exist to make auditing cheap, not to make the judgement unnecessary.

6 Related work

Comparison against an independent implementation is long-standing practice in tax-benefit microsimulation. EUROMOD validates its baselines against external aggregates and, where available, national models, and documents the results per country (Sutherland and Figari 2013); TAXSIM has served a similar reference role for United States federal and state income tax since 1993 (Feenberg and Coutts 1993). What that literature generally reports is a comparison of aggregates or a discussion of discrepancies in prose. Our contribution is narrower and more mechanical: a committed, schema-validated classification of every case-level difference, with the evidence recomputed by the build, and a predicate that depends on the classification rather than on a rate.

In software engineering, the technique is differential testing (McKeeman 1998), and the underlying difficulty is the test oracle problem (Barr et al. 2015). The dispositions mechanism can be read as a structured, persistent answer to what that literature calls the human oracle cost: rather than re-adjudicating the same known divergence on every run, the adjudication is recorded once, with its evidence, and expires automatically when the values it explains move.

On the encoding side, the Rules as Code programme argues that governments should publish machine-consumable versions of legislation alongside the natural-language text (Mohun and Roberts 2020), and Catala demonstrates that statutory text can be translated into an executable form with a systematic, auditable correspondence to its source (Merigoux et al. 2021). Both are concerned with getting law into code faithfully. This paper concerns the step after: how the resulting artefact is held to account against something that already exists, and what may honestly be published about the result.

7 Availability

The harness, the universes, the dispositions files, the committed comparison reports, the ratchets, and the scoreboard are public. The predicate is evaluated by the same script that writes the scoreboard, and the check that fails a build when the committed scoreboard drifts from a recomputation is the check a reader can run.

The four jurisdiction rows in this paper can be reproduced from a clone by reading the committed scoreboard, and every classification behind them can be re-derived by re-running the dispositions check, which parses each entry’s arithmetic and evaluates it against the live report.

8 References

- Barr, Earl T., Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. “The Oracle Problem in Software Testing: A Survey.” *IEEE Transactions on Software Engineering* 41 (5): 507–25. <https://doi.org/10.1109/TSE.2014.2372785>.
- Feenberg, Daniel, and Elisabeth Coutts. 1993. “An Introduction to the TAXSIM Model.” *Journal of Policy Analysis and Management* 12 (1): 189–94. <https://doi.org/10.2307/3325474>.

- McKeeman, William M. 1998. “Differential Testing for Software.” *Digital Technical Journal* 10 (1): 100–107.
- Merigoux, Denis, Nicolas Chataing, and Jonathan Protzenko. 2021. “Catala: A Programming Language for the Law.” *Proceedings of the ACM on Programming Languages* 5 (ICFP): 77:1–29. <https://doi.org/10.1145/3473582>.
- Mohun, James, and Alex Roberts. 2020. *Cracking the Code: Rulemaking for Humans and Machines*. OECD Working Papers on Public Governance No. 42. OECD. <https://doi.org/10.1787/3afe6ba5-en>.
- Sutherland, Holly, and Francesco Figari. 2013. “EUROMOD: The European Union Tax-Benefit Microsimulation Model.” *International Journal of Microsimulation* 6 (1): 4–26. <https://doi.org/10.34196/ijm.00075>.